Competitive Programming Mastery

From Zero to Hero

Complete Guide with 150+ Problems

Ultimate CP Resource

March 7, 2026

Contents

Chapter 1

CP Fundamentals — Shuruwaat

1.1 CP Kya Hai?

Competitive Programming Definition

Competitive Programming (CP) = Algorithmic problems ko **time limit** mein solve karna using **efficient code**.
3 Pillars:

1. **Algorithm Design** — Problem todna, approach sochna
2. **Data Structures** — Sahi tool chunna (STL mastery)
3. **Implementation** — Clean, fast, bug-free code

1.2 Ultimate Template

Listing 1.1: ultimate_template.cpp — Har contest ke liye ready

```cpp
#include <bits/stdc++.h>
using namespace std;

// Type Aliases
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vll = vector<ll>;
using vvi = vector<vector<int>>;
using vs = vector<string>;

// Constants
const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;
```

```cpp
const double PI = acos(-1.0);
const double EPS = 1e-9;

// Fast I/O
void fastIO() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
}

// Debug (local mein 1, submission mein 0)
#define DEBUG 0
#if DEBUG
    #define debug(x) cerr << #x << " = " << x << "\n"
    #define debug2(x,y) cerr << #x << "=" << x << ", " << #y << "="
        << y << "\n"
#else
    #define debug(x)
    #define debug2(x,y)
#endif

// Utility Functions
template<typename T>
void print(const vector<T>& v, string sep = " ") {
    for (int i = 0; i < v.size(); i++)
        cout << v[i] << (i == v.size()-1 ? "\n" : sep);
}

template<typename T>
void read(vector<T>& v) {
    for (auto& x : v) cin >> x;
}

// Modular Arithmetic
ll modPow(ll base, ll exp, ll mod = MOD) {
    ll res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return res;
}

ll modInv(ll a, ll mod = MOD) {
    return modPow(a, mod - 2, mod);
}

// GCD, LCM
ll gcd(ll a, ll b) { return b == 0 ? a : gcd(b, a % b); }
```

```cpp
ll lcm(ll a, ll b) { return (a / gcd(a, b)) * b; }

void solve() {
    // Solution yahan likho
    int n; cin >> n;
    vi a(n); read(a);
    ll sum = accumulate(a.begin(), a.end(), 0LL);
    cout << sum << "\n";
}

int main() {
    fastIO();
    int t = 1;
    cin >> t;
    while (t--) solve();
    return 0;
}
```

Common Mistakes

- **endl vs \n:** `endl` flush karta hai, slow hai. Hamesha `\n` use karo.
- **long long:** Sum, product mein overflow hota hai. `int` ki jagah `ll` use karo.
- **MOD operations:** Negative numbers ka dhyan rakhna. `(x%MOD + MOD)%MOD` safe hai.

Chapter 2

Time Complexity — Sabse Bada Dost

Big-O Complexity Chart

Complexity	Name	Max n (1 sec)
$O(1)$	Constant	Any
$O(\log n)$	Logarithmic	10^{18}
$O(\sqrt{n})$	Square root	10^{14}
$O(n)$	Linear	10^7
$O(n \log n)$	Linearithmic	10^6
$O(n^2)$	Quadratic	10^4
$O(n^3)$	Cubic	500
$O(2^n)$	Exponential	25
$O(n!)$	Factorial	10

Rule: 10^8 operations $\approx$ 1 second

Listing 2.1: Complexity Examples

```
// O(n) - Linear
for (int i = 0; i < n; i++) cout << i << "\n";

// O(n^2) - Quadratic
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) { /* ... */ }

// O(n log n) - Sorting
sort(a.begin(), a.end());

// O(log n) - Binary Search
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < target) lo = mid + 1;
    else hi = mid;
}
```

Chapter 3

Pattern 1: Array Manipulation

Pattern: Two Pointers

When to use:

- Subarray problems
- Pair finding
- Running sum/mean

3.1 Problem: Two Sum

Problem: Two Sum — LeetCode 1

Problem: Array `nums` aur target diya hai. Do numbers find karo jinka sum target ke barabar ho.
Input: `nums = [2,7,11,15], target = 9`
Output: `[0,1]` (kyunki $2 + 7 = 9$)

Listing 3.1: Two Sum — 3 Approaches

```
// Approach 1: Brute Force      O(n^2)
vector<int> twoSumBrute(vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (nums[i] + nums[j] == target)
                return {i, j};
    return {};
}

// Approach 2: Hash Map      O(n) time, O(n) space [BEST]
vector<int> twoSumOptimal(vector<int>& nums, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement))
```

```
            return {seen[complement], i};
        seen[nums[i]] = i;
    }
    return {};
}

// Approach 3: Two Pointers (sorted array ke liye)     O(n log n)
vector<int> twoSumSorted(vector<int>& nums, int target) {
    vector<pair<int, int>> indexed;
    for (int i = 0; i < nums.size(); i++)
        indexed.push_back({nums[i], i});
    sort(indexed.begin(), indexed.end());

    int left = 0, right = indexed.size() - 1;
    while (left < right) {
        int sum = indexed[left].first + indexed[right].first;
        if (sum == target)
            return {indexed[left].second, indexed[right].second};
        else if (sum < target) left++;
        else right--;
    }
    return {};
}
```

Key Insights

1. **Hash map = O(n):** Pehle dekha hua store karo, future mein check karo
2. **Complement formula:** `complement = target - current`
3. **Two pointers:** Sorted array mein efficient

3.2 Problem: Maximum Subarray (Kadane's Algorithm)

Problem: Maximum Subarray — LeetCode 53

Problem: Contiguous subarray with largest sum find karo.
Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6 (subarray `[4,-1,2,1]`)

Listing 3.2: Kadane's Algorithm — O(n)

```
// KADANE'S ALGORITHM
int maxSubArray(vector<int>& nums) {
    int currentSum = nums[0];
    int maxSum = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        // Decision: start fresh OR extend previous
        currentSum = max(nums[i], currentSum + nums[i]);
```

```
        maxSum = max(maxSum, currentSum);
    }
    return maxSum;
}

// With indices tracking
int maxSubArrayWithIndices(vector<int>& nums, int& start, int& end)
    {
    int currentSum = nums[0], maxSum = nums[0];
    int currentStart = 0;
    start = 0; end = 0;

    for (int i = 1; i < nums.size(); i++) {
        if (currentSum + nums[i] < nums[i]) {
            currentSum = nums[i];
            currentStart = i;
        } else {
            currentSum += nums[i];
        }

        if (currentSum > maxSum) {
            maxSum = currentSum;
            start = currentStart;
            end = i;
        }
    }
    return maxSum;
}
```

Pattern: Kadane's Pattern

Use when: Maximum/minimum sum of contiguous subarray.
Core idea: Har position pe decide karo — **extend** ya **start fresh**?
Formula: `dp[i] = max(arr[i], dp[i-1] + arr[i])`

3.3 Problem: Sliding Window Maximum

Problem: Sliding Window Maximum — LeetCode 239

Problem: Array `nums` aur window size `k`. Har window ka maximum nikaalo.
Input: `nums = [1,3,-1,-3,5,3,6,7], k = 3`
Output: `[3,3,5,5,6,7]`

Listing 3.3: Sliding Window Maximum — Deque O(n)

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    vector<int> result;
    deque<int> dq;  // Store indices, front = max of current window

    for (int i = 0; i < nums.size(); i++) {
```

```
        // Remove elements outside window
        if (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();

        // Remove smaller elements (useless for future)
        while (!dq.empty() && nums[dq.back()] <= nums[i])
            dq.pop_back();

        dq.push_back(i);

        // Window formed, record max
        if (i >= k - 1)
            result.push_back(nums[dq.front()]);
    }
    return result;
}
```

Sliding Window Tricks

1. **Deque = O(n):** Har element ek baar push, ek baar pop
2. **Monotonic queue:** Decreasing order maintain karo
3. **Index store karo:** Value nahi, index store karo

Chapter 4

Pattern 2: Binary Search

Pattern: Binary Search Variants

Condition: Monotonic function (sorted array)
Template:

1. Search space [lo, hi] define karo
2. while (lo < hi)
3. mid = lo + (hi - lo) / 2 (overflow safe)
4. Condition check, space shrink karo

4.1 Binary Search Templates

Listing 4.1: 5 Binary Search Variants

```
// Variant 1: Classic Binary Search
int binarySearch(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

// Variant 2: Lower Bound (first >= target) [MOST USEFUL]
int lowerBound(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid;
    }
```

```
    return lo;
}

// Variant 3: Upper Bound (first > target)
int upperBound(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// Variant 4: Search in Rotated Sorted Array
int searchRotated(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;

        if (nums[lo] <= nums[mid]) {  // Left half sorted
            if (nums[lo] <= target && target < nums[mid])
                hi = mid - 1;
            else
                lo = mid + 1;
        } else {  // Right half sorted
            if (nums[mid] < target && target <= nums[hi])
                lo = mid + 1;
            else
                hi = mid - 1;
        }
    }
    return -1;
}

// Variant 5: Find Peak Element
int findPeakElement(vector<int>& nums) {
    int lo = 0, hi = nums.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[mid + 1]) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

4.2 Problem: Aggressive Cows

Problem: Aggressive Cows — Binary Search on Answer

Problem: N stalls, C cows. Minimum distance maximize karo.
Key Insight: "Can we place cows with at least X distance?" pe binary search.

Listing 4.2: Aggressive Cows — Binary Search on Answer

```
bool canPlaceCows(vector<int>& pos, int cows, int minDist) {
    int placed = 1, lastPos = pos[0];
    for (int i = 1; i < pos.size(); i++) {
        if (pos[i] - lastPos >= minDist) {
            placed++;
            lastPos = pos[i];
            if (placed == cows) return true;
        }
    }
    return false;
}

int aggressiveCows(vector<int>& pos, int cows) {
    sort(pos.begin(), pos.end());
    int lo = 1, hi = pos.back() - pos[0], ans = 0;

    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (canPlaceCows(pos, cows, mid)) {
            ans = mid;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return ans;
}
```

Pattern: Binary Search on Answer

When to use: "Maximize minimum" ya "Minimize maximum" problems
Examples:

- Painter's Partition
- Book Allocation
- Koko Eating Bananas
- Magnetic Force Between Balls

Chapter 5

Pattern 3: Dynamic Programming

DP Fundamentals

Dynamic Programming = Recursion + Memoization + Tabulation
3 Steps:

1. **State:** `dp[i]` kya represent karta hai?
2. **Recurrence:** `dp[i]` kaise calculate hoga?
3. **Base case:** Chhote inputs ke liye direct answer

5.1 Problem: Climbing Stairs

Listing 5.1: Climbing Stairs — 4 Approaches

```
// Problem: N stairs, 1 ya 2 steps chadh sakte ho. Kitne ways?

// Approach 1: Recursion      O(2^n) TLE
int climbRecursive(int n) {
    if (n <= 2) return n;
    return climbRecursive(n-1) + climbRecursive(n-2);
}

// Approach 2: Memoization      O(n)
int climbMemo(int n, vector<int>& dp) {
    if (n <= 2) return n;
    if (dp[n] != -1) return dp[n];
    return dp[n] = climbMemo(n-1, dp) + climbMemo(n-2, dp);
}

// Approach 3: Tabulation      O(n), O(n) space
int climbTab(int n) {
    if (n <= 2) return n;
    vector<int> dp(n+1);
    dp[1] = 1; dp[2] = 2;
    for (int i = 3; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
```

```
    return dp[n];
}

// Approach 4: Space Optimized      O(n), O(1) space [BEST]
int climbOptimal(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

5.2 Problem: 0/1 Knapsack

Listing 5.2: 0/1 Knapsack — Complete Solution

```
// 2D DP      O(n*W)
int knapsack01(vector<int>& weight, vector<int>& value, int W) {
    int n = weight.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w];  // Don't take
            if (w >= weight[i-1])
                dp[i][w] = max(dp[i][w],
                    value[i-1] + dp[i-1][w - weight[i-1]]);
        }
    }
    return dp[n][W];
}

// 1D Space Optimized      O(W) space
int knapsackOptimized(vector<int>& weight, vector<int>& value, int
    W) {
    int n = weight.size();
    vector<int> dp(W+1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= weight[i]; w--)  // REVERSE!
            dp[w] = max(dp[w], value[i] + dp[w - weight[i]]);
    }
    return dp[W];
}

// Unbounded Knapsack (infinite items)      FORWARD iterate
```

```
int unboundedKnapsack(vector<int>& weight, vector<int>& value, int
    W) {
    vector<int> dp(W+1, 0);
    for (int i = 0; i < weight.size(); i++)
        for (int w = weight[i]; w <= W; w++)  // FORWARD!
            dp[w] = max(dp[w], value[i] + dp[w - weight[i]]);
    return dp[W];
}
```

5.3 Problem: Longest Common Subsequence

Listing 5.3: LCS — String DP

```
// Tabulation      O(n*m)
int longestCommonSubsequence(string s1, string s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i-1] == s2[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[n][m];
}

// Space Optimized      O(min(n,m))
int lcsSpaceOptimized(string s1, string s2) {
    if (s1.size() < s2.size()) swap(s1, s2);
    int n = s1.size(), m = s2.size();
    vector<int> prev(m+1, 0), curr(m+1, 0);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i-1] == s2[j-1])
                curr[j] = 1 + prev[j-1];
            else
                curr[j] = max(prev[j], curr[j-1]);
        }
        swap(prev, curr);
    }
    return prev[m];
}

// Print LCS
string getLCS(string s1, string s2) {
    int n = s1.size(), m = s2.size();
```

```
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));

    // Fill DP
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            if (s1[i-1] == s2[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

    // Backtrack
    string lcs;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i-1] == s2[j-1]) {
            lcs.push_back(s1[i-1]);
            i--; j--;
        } else if (dp[i-1][j] > dp[i][j-1])
            i--;
        else
            j--;
    }
    reverse(lcs.begin(), lcs.end());
    return lcs;
}
```

Chapter 6

Pattern 4: Graph Algorithms

6.1 Graph Representation

Listing 6.1: Graph Representation Methods

```
// Adjacency List      O(V+E) space [MOST COMMON]
class Graph {
    int n;
    vector<vector<pair<int,int>>> adj; // (neighbor, weight)

public:
    Graph(int n) : n(n), adj(n) {}

    void addEdge(int u, int v, int w = 1, bool undirected = true) {
        adj[u].push_back({v, w});
        if (undirected) adj[v].push_back({u, w});
    }
};

// Edge List      For Kruskal
struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const { return w < other.w; }
};
```

6.2 BFS and DFS

Listing 6.2: BFS and DFS Complete

```
// BFS      Shortest path in unweighted graph
vector<int> bfs(int start, int n, vector<vector<int>>& adj) {
    vector<int> dist(n, -1), parent(n, -1);
    queue<int> q;

    dist[start] = 0;
    q.push(start);

```

```
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }
    return dist;
}

// DFS      Connected components
void dfs(int u, vector<vector<int>>& adj, vector<bool>& vis, vector
    <int>& comp) {
    vis[u] = true;
    comp.push_back(u);
    for (int v : adj[u])
        if (!vis[v]) dfs(v, adj, vis, comp);
}

// Cycle detection in undirected graph
bool hasCycleUndirected(int n, vector<vector<int>>& adj) {
    vector<bool> vis(n, false);
    function<bool(int,int)> dfs = [&](int u, int p) -> bool {
        vis[u] = true;
        for (int v : adj[u]) {
            if (!vis[v]) { if (dfs(v, u)) return true; }
            else if (v != p) return true;
        }
        return false;
    };
    for (int i = 0; i < n; i++)
        if (!vis[i] && dfs(i, -1)) return true;
    return false;
}
```

6.3 Shortest Path Algorithms

Listing 6.3: Dijkstra and Bellman-Ford

```
// Dijkstra      O((V+E) log V)
vector<long long> dijkstra(int start, int n, vector<vector<pair<int
    ,int>>>& adj) {
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair
        <ll,int>>> pq;

    dist[start] = 0;
```

```
    pq.push({0, start});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

// Bellman-Ford      O(V*E), handles negative weights
vector<long long> bellmanFord(int start, int n, vector<Edge>& edges
    ) {
    vector<long long> dist(n, LLONG_MAX);
    dist[start] = 0;

    for (int i = 0; i < n-1; i++)
        for (auto& e : edges)
            if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.
                v])
                dist[e.v] = dist[e.u] + e.w;

    // Negative cycle check
    for (auto& e : edges)
        if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
            return {}; // Negative cycle detected

    return dist;
}
```

6.4 Minimum Spanning Tree

Listing 6.4: Kruskal and Prim's Algorithm

```
// Union-Find for Kruskal
class UnionFind {
    vector<int> parent, rank_;
public:
    UnionFind(int n) {
        parent.resize(n); rank_.resize(n);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
```

```
        return parent[x];
    }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (rank_[x] < rank_[y]) swap(x, y);
        parent[y] = x;
        if (rank_[x] == rank_[y]) rank_[x]++;
        return true;
    }
};

// Kruskal      O(E log E)
long long kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    UnionFind uf(n);
    long long mstWeight = 0;
    int edgesUsed = 0;

    for (auto& e : edges) {
        if (uf.unite(e.u, e.v)) {
            mstWeight += e.w;
            if (++edgesUsed == n-1) break;
        }
    }
    return (edgesUsed == n-1) ? mstWeight : -1;
}

// Prim's      O((V+E) log V)
long long prim(int start, int n, vector<vector<pair<int,int>>>& adj
    ) {
    vector<bool> inMST(n, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<
        pair<int,int>>> pq;
    long long mstWeight = 0;
    int edgesUsed = 0;

    pq.push({0, start});
    while (!pq.empty() && edgesUsed < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (inMST[u]) continue;

        inMST[u] = true;
        mstWeight += w;
        edgesUsed++;

        for (auto [v, w2] : adj[u])
            if (!inMST[v]) pq.push({w2, v});
    }
    return (edgesUsed == n) ? mstWeight : -1;
}
```

Chapter 7

Pattern 5: Trees and Advanced DS

7.1 Binary Tree Operations

Listing 7.1: Binary Tree Complete Template

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// Inorder traversal
void inorder(TreeNode* root, vector<int>& res) {
    if (!root) return;
    inorder(root->left, res);
    res.push_back(root->val);
    inorder(root->right, res);
}

// Level order (BFS)
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
```

```
}

// LCA
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
    TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;
    return left ? left : right;
}
```

7.2 Segment Tree

Listing 7.2: Segment Tree with Lazy Propagation

```
class SegmentTree {
    vector<int> tree, lazy;
    int n;

    void push(int node, int start, int end) {
        if (lazy[node] != 0) {
            tree[node] += lazy[node] * (end - start + 1);
            if (start != end) {
                lazy[2*node] += lazy[node];
                lazy[2*node+1] += lazy[node];
            }
            lazy[node] = 0;
        }
    }

    void updateRange(int node, int start, int end, int l, int r,
        int val) {
        push(node, start, end);
        if (r < start || end < l) return;
        if (l <= start && end <= r) {
            lazy[node] += val;
            push(node, start, end);
            return;
        }
        int mid = (start + end) / 2;
        updateRange(2*node, start, mid, l, r, val);
        updateRange(2*node+1, mid+1, end, l, r, val);
        tree[node] = tree[2*node] + tree[2*node+1];
    }

    int queryRange(int node, int start, int end, int l, int r) {
        push(node, start, end);
        if (r < start || end < l) return 0;
        if (l <= start && end <= r) return tree[node];
```

```
        int mid = (start + end) / 2;
        return queryRange(2*node, start, mid, l, r) +
               queryRange(2*node+1, mid+1, end, l, r);
    }

public:
    SegmentTree(int n) : n(n) {
        tree.resize(4*n);
        lazy.resize(4*n);
    }

    void updateRange(int l, int r, int val) {
        updateRange(1, 0, n-1, l, r, val);
    }

    int queryRange(int l, int r) {
        return queryRange(1, 0, n-1, l, r);
    }
};
```

Chapter 8

Complete Problem List — 150+ Questions

8.1 Level 1: Fundamentals (Days 1-15)

Array & Vector Problems

1. Two Sum (1) — Hash map
2. Best Time to Buy/Sell Stock (121) — Kadane
3. Contains Duplicate (217) — Set
4. Product of Array Except Self (238) — Prefix/Suffix
5. Maximum Subarray (53) — Kadane's algorithm
6. Maximum Product Subarray (152) — Modified Kadane
7. Find Minimum in Rotated Sorted Array (153) — Binary search
8. Search in Rotated Sorted Array (33) — Binary search
9. 3Sum (15) — Two pointers
10. Container With Most Water (11) — Two pointers

Two Pointers & Sliding Window

11. Valid Palindrome (125)
12. Reverse String (344)
13. Move Zeroes (283)
14. Remove Duplicates from Sorted Array (26)
15. Longest Substring Without Repeating Characters (3)
16. Longest Repeating Character Replacement (424)

17. Permutation in String (567)
18. Find All Anagrams (438)
19. Minimum Window Substring (76)

8.2 Level 2: Intermediate (Days 16-45)

Binary Search Deep Dive

20. Binary Search (704)
21. First Bad Version (278)
22. Search Insert Position (35)
23. Find First and Last Position (34)
24. Search a 2D Matrix (74)
25. Find Peak Element (162)
26. Koko Eating Bananas (875)
27. Capacity To Ship Packages (1011)
28. Split Array Largest Sum (410)
29. Aggressive Cows (SPOJ)

Dynamic Programming Classics

30. Climbing Stairs (70)
31. House Robber (198)
32. House Robber II (213)
33. Longest Increasing Subsequence (300)
34. Longest Common Subsequence (1143)
35. Edit Distance (72)
36. Word Break (139)
37. Unique Paths (62)
38. Minimum Path Sum (64)
39. Coin Change (322)
40. Coin Change 2 (518)

41. Partition Equal Subset Sum (416)
42. Target Sum (494)

8.3 Level 3: Advanced (Days 46-75)

Graph Algorithms

43. Number of Islands (200) — BFS/DFS
44. Clone Graph (133)
45. Course Schedule (207) — Topological sort
46. Course Schedule II (210)
47. Pacific Atlantic Water Flow (417)
48. Word Ladder (127)
49. Network Delay Time (743) — Dijkstra
50. Cheapest Flights Within K Stops (787)
51. Critical Connections in Network (1192) — Tarjan

Backtracking

52. Subsets (78)
53. Permutations (46)
54. Combinations (77)
55. Combination Sum (39)
56. N-Queens (51)
57. Sudoku Solver (37)

Chapter 9

STL Mastery Cheat Sheet

Listing 9.1: STL Quick Reference

```
// Vectors
vector<int> v;
v.push_back(x); v.pop_back();
v.insert(v.begin()+i, x);
v.erase(v.begin()+i);
sort(v.begin(), v.end());
auto it = lower_bound(v.begin(), v.end(), x);

// Set / Map
set<int> s; s.insert(x); s.erase(x);
if (s.count(x)) // exists
map<string, int> m;
m["key"] = value;
for (auto& [k, v] : m) { }

// Priority Queue
priority_queue<int> pq; // max heap
priority_queue<int, vector<int>, greater<int>> minPQ;

// Algorithms
*min_element(v.begin(), v.end());
*max_element(v.begin(), v.end());
accumulate(v.begin(), v.end(), 0LL);
reverse(v.begin(), v.end());
next_permutation(v.begin(), v.end());
```

Chapter 10

Final Tips & Strategy

100-Day Roadmap

Days	Focus	Problems/Day
1-15	Arrays, Two Pointers, STL	3-4
16-30	Binary Search, Linked List	3
31-45	Trees, BST, Heaps	3
46-60	Dynamic Programming	2-3
61-75	Graph Algorithms	2-3
76-90	Backtracking, Advanced	2
91-100	Mixed Practice, Contests	Contest + Review

Golden Rules

1. **Consistency** > Intensity — Daily 3-4 problems
2. Pehle **brute force**, phir optimize
3. Har problem ka **pattern** notebook mein likho
4. **Editorial** padhna seekho — 30 min mein stuck ho to dekh lo
5. **Virtual contests** do — Codeforces, LeetCode

Best of Luck!

Now go and crack those CP problems!